

ComfyUI Wan2.2 workflow

专有名词功能逻辑深度解析

适用环境: ComfyUI-aki-v3 + RTX 5070 Ti (16GB VRAM)

核心模型: Wan2.2 T2V / I2V 14B + NSFW LoRA

生成日期: 2026-05-07

本文档从底层数学原理出发, 逐项解析 ComfyUI Wan2.2 workflow 中

每个专有名词的功能逻辑、设计动机与参数调优策略。

面向需要深入理解模型机制的高级用户与开发者。

ComfyUI Wan2.2 工作流：专有名词功能逻辑可核验版

更新日期：2026-05-08

适用环境：ComfyUI-aki-v3

+

ComfyUI-WanVideoWrapper

文档目标：把容易误读的公式改成可读、可核验、能对照源码的说明。

重要修订：删除

PDF

中无法正常排版的

LaTeX

公式，改用代码块公式、变量解释和源码依据。

0. 可信度标记与核验方法

本文把结论分为三类：

标记	含义	使用方式
源码可核验	本机源码里能直接看到等价计算	可按文档给出的文件路径搜索对应代码
工作流可核验	本机工作流/测试脚本里能看到模型、节点、参数连接	可打开对应 JSON 或 Python 测试脚本确认
背景知识	属于扩散模型/LoRA/CFG 的通用理论，不一定是本机代码只用于帮助理解，不作为“本机一定如此执行”的证据	

本次重点核验过的本机源码：

内容	核验文件
Flow Matching 的 σ 生成、shift 重映射、add_noise、step 更新	<code>`ComfyUI/custom_nodes/ComfyUI-WanVideoWrapper/wanvideo/schedulers/basic_flowmat</code>
WanVideoSampler 的 cfg 参数、正/负分支推理、最终 CFG 合成	<code>`ComfyUI/custom_nodes/ComfyUI-WanVideoWrapper/nodes_sampler.py`</code>
LoRA strength 如何参与权重增量计算	<code>`ComfyUI/custom_nodes/ComfyUI-WanVideoWrapper/custom_linear.py`</code>
T2V 测试脚本里的实际参数	<code>`agent-skills/temp/nsfw_t2v_test.py`</code>
T2V API 工作流 JSON	<code>`agent-skills/temp/nsfw_t2v_api_prompt.json`</code>

阅读建议：

- 看到“公式”时，优先看代码块里的伪代码表达，它和源码更接近。
- 看到“背景知识”时，把它当成理解模型的辅助解释，不要当成本机源码证据。
- 看到“调参建议”时，它是根据源码机制、当前工作流参数和常见采样经验综合推断，不是数学定理。

1. Flow Matching: WanVideoWrapper 里能核验的部分

1.1 它在这里到底做什么

可信度：源码可核验

在本机 WanVideoWrapper 中，Flow Matching 相关逻辑主要由调度器维护。它关心三件事：

- 当前采样一共有多少步。
- 每一步对应的噪声水平 σ 是多少。
- 模型输出 `model_output` 以后，如何把当前 latent 往下一步推进。

源码中的核心更新规则是：

```
prev_sample = sample + model_output * (sigma_next - sigma_cur..
```

对应源码位置：basic_flowmatch.py 的 step() 函数。

变量解释：

变量	含义
<code>`sample`</code>	当前步骤的 latent，也就是还带噪声的视频潜变量
<code>`model_output`</code>	模型预测出来的“应该怎么移动”的方向，常称 velocity 或 flow prediction
<code>`sigma_current`</code>	当前步骤噪声水平
<code>`sigma_next`</code>	下一步骤噪声水平
<code>`prev_sample`</code>	更新后的 latent，噪声更少，离最终视频更近

这不是传统 DDPM 里“直接预测噪声 epsilon 后按 alpha/beta 反推”的写法。本机这条 WanVideoWrapper 路径更接近“沿着模型预测的方向场，把 latent 从高噪声位置推向低噪声位置”。

1.2 add_noise 的真实公式

可信度：源码可核验

源码中的 `add_noise()` 用的是线性混合：

```
sample = (1 - sigma) * original_samples + sigma * noise
```

变量解释：

变量	含义
<code>`original_samples`</code>	干净 latent，通常来自原图/原视频编码结果
<code>`noise`</code>	随机高斯噪声
<code>`sigma`</code>	当前噪声比例，越大越接近纯噪声
<code>`sample`</code>	加噪后的 latent

直观理解：

```
sigma = 0.0 -> sample = 原始 latent
sigma = 0.5 -> sample = 50% 原始 latent + 50% 噪声
sigma = 1.0 -> sample = 纯噪声
```

这条公式能直接解释为什么 `denoise_strength`、`start_step`、`add_noise_to_samples` 会影响图生视频/视频转视频保留原图的程度。

1.3 shift 参数的真实公式

可信度：源码可核验

源码中的 `sigma` 先线性生成，再用 `shift` 重映射：

```
sigma_shifted = shift * sigma / (1 + (shift - 1) * sigma)
```

对应源码位置：`basicflowmatch.py` 的 `set_timesteps()` 函数。

变量解释：

变量	含义
<code>`sigma`</code>	原始线性噪声水平
<code>`shift`</code>	用户输入的 shift 参数
<code>`sigma_shifted`</code>	最终用于采样器的噪声水平

重要结论：

- ``shift = 1`` 时，``sigma_shifted = sigma``，不改变调度。
- ``shift > 1`` 时，中间区间的 `sigma` 会被整体抬高。
- `sigma` 被抬高意味着采样过程在“较高噪声状态”停留更久，细节阶段会更集中在末尾少数步骤。

1.4 按本机公式重新计算的 20 步 sigma 表

可信度：源码公式 + 本机 Python 计算

计算条件：

```
steps = 20
sigma_min = 0.003 / 1.002
sigma_max = 1.0
denoising_strength = 1.0
```

抽样结果如下：

步序号	shift=3	shift=5	shift=8	shift=12
0	1.0000	1.0000	1.0000	1.0000
4	0.9187	0.9495	0.9679	0.9783
8	0.8057	0.8736	0.9171	0.9431
12	0.6382	0.7462	0.8247	0.8759
16	0.3644	0.4886	0.6045	0.6963
19	0.0089	0.0148	0.0235	0.0348

低噪声区间步数：

shift	sigma < 0.6	sigma < 0.4	sigma < 0.2	sigma < 0.1
3	7 步	4 步	2 步	1 步
5	5 步	3 步	1 步	1 步
8	3 步	2 步	1 步	1 步
12	3 步	1 步	1 步	1 步

修正说明：旧版文档里 shift 表的 sigma 数值偏低，本版已按本机源码公式重算。更准确的表述不是“shift=8 只有 4 步细节”，而是：

```
shift 越高，sigma 保持在高噪声区间的步数越多；
低噪声、细节修正的有效窗口越集中在采样末尾。
```

2. CFG：正负提示词如何进入采样

2.1 WanVideoSampler 里的真实 CFG 公式

可信度：源码可核验
源码中普通路径的最终 CFG 合成是：

```
noise_pred = noise_pred_uncond_text + cfg_scale * (noise_pred..
```

对应源码位置：nodesampler.py 的 predictwith_cfg() 函数。
变量解释：

变量	含义
`noise_pred_cond`	带正向提示词时模型预测的更新方向
`noise_pred_uncond_text`	带负向提示词时模型预测的更新方向
`cfg_scale`	K采样器/WanVideoSampler 里的 CFG 参数
`noise_pred`	最终送给调度器 step() 的模型输出

几个关键值：

```
cfg = 1.0:
noise_pred = cond

cfg = 2.0:
noise_pred = uncond + 2 * (cond - uncond)
           = 2 * cond - uncond

cfg = 5.0:
```

```
noise_pred = uncond + 5 * (cond - uncond)
            = 5 * cond - 4 * uncond
```

2.2 为什么高 CFG 可能让画面更“硬”

可信度：源码公式 + 采样经验推断

从上面的公式可见，CFG 越高， $\text{cond} - \text{uncond}$ 这个方向差值被放大得越厉害。

这会带来两类效果：

CFG 趋势	常见变化
低 CFG	更自然、更自由，提示词遵循度弱一些
中 CFG	提示词可控和自然度相对平衡
高 CFG	更强行靠近提示词，但容易过饱和、边缘硬、动作僵、伪影变多

对于你当前这种带 LoRA 的视频工作流，高 CFG 不一定等于更强 LoRA 效果。因为 CFG 放大的不是“LoRA 本身”，而是正向分支和负向分支之间的差值。差值被过度放大后，可能出现过冲、纹理不稳定、动作不自然等问题。

2.3 为什么文档不再写“高 CFG 必然放大安全偏置”

可信度：谨慎修正

旧版说法过于绝对。源码只能证明 CFG 的合成公式，不能直接证明“安全偏置一定如何被放大”。更稳妥的说法是：

高 CFG 会强烈放大正向分支与负向分支的差异。
如果基础模型、负向提示词或提示词语义本身把结果推向保守/干净/安全方向，
高 CFG 可能让这种趋势更明显。

调参上仍建议先试：

```
cfg = 2.0 ~ 3.0
```

但原因应表述为“减少过冲和过约束，提高 LoRA

与模型自由生成空间的平衡”，而不是把所有问题都归因于一个无法从源码直接证明的“安全偏置公式”。

3. LoRA strength：模型强度正负值到底做了什么

3.1 LoRA 的真实加权方式

可信度：源码可核验

本机源码中标准 LoRA 的核心计算是：

```
patch_diff = lora_diff_0 @ lora_diff_1
alpha = lora_diff_2 / rank # 如果 lora_diff_2 不为 0
scale = lora_strength * alpha
new_weight = base_weight + patch_diff * scale
```

如果 LoRA 已经是单个完整差分张量，则是：

```
new_weight = base_weight + lora_diff * lora_strength
```

对应源码位置：customlinear.py 的 applylora() 和 applysinglelora()。

3.2 strength 为正数

可信度：源码可核验 + 使用经验

正数表示沿 LoRA 学到的方向修改基础模型权重：

strength = 0.0 → 不使用 LoRA 增量
strength = 0.5 → 使用一半 LoRA 增量
strength = 1.0 → 使用完整 LoRA 增量
strength = 1.2 → 超过训练时的标准强度，可能更明显，也更容易崩

建议区间：

LoRA 用途	推荐起点	常见上限
加速 LoRA, 如 lightx2v	0.9	1.0
风格 LoRA	0.5	0.9
角色/人物 LoRA	0.6	0.9
强内容倾向 LoRA	0.6	0.9

3.3 strength 为负数

可信度：源码可核验 + 使用风险推断

负数表示从基础权重中减去 LoRA 学到的方向：

```
strength = -0.5 -> base_weight - 0.5 * lora_diff
strength = -1.0 -> base_weight - 1.0 * lora_diff
```

它不是“把提示词变成反义词”，也不是“自动生成相反内容”。它只是权重空间里的反向推力。

适合尝试的场景：

- 去掉某种风格味道，例如把过强的动漫/油画风格往回拉。
- 抑制一个 LoRA 里不想要的构图习惯。

不建议的场景：

- 对人体结构类、动作类、强内容类 LoRA 大幅使用负值。
- 因为负值可能把模型推到训练分布之外，结果可能是比例异常、动作破碎、局部结构奇怪。

3.4 多 LoRA 的叠加逻辑

可信度：源码可核验

多个 LoRA patch 最终会作为多个增量逐个叠到对应线性层上。简化表达：

```
new_weight = base_weight
new_weight += lora_A_diff * strength_A
new_weight += lora_B_diff * strength_B
new_weight += lora_C_diff * strength_C
```

所以多 LoRA 不是“谁覆盖谁”的简单关系，而是在许多线性层权重上叠加多个方向。实际画面中出现冲突，通常是因为不同 LoRA 对同一批视觉特征提出了不同倾向。

4. K采样器 / WanVideoSampler：各参数的功能逻辑

4.1 steps

可信度：源码可核验 + 采样经验

steps 决定调度器生成多少个 timestep/sigma，也就是采样迭代次数。

```
steps 越少 -> 更快，细节修正机会少
steps 越多 -> 更慢，细节和稳定性可能提升，但收益递减
```

对当前 fp8 视频模型，建议先从：

```
steps = 16 ~ 24
```

开始测试。不要盲目拉到很高，因为长视频、fp8 量化、LoRA 叠加都会让高步数的收益变得不稳定。

4.2 cfg

可信度：源码可核验

作用：控制正向提示词分支相对负向提示词分支的外推强度。

推荐测试顺序：

1.8 -> 2.2 -> 2.6 -> 3.0 -> 3.5

如果画面变硬、闪烁、细节反而少，优先降 CFG。

4.3 shift

可信度：源码可核验
作用：改写 sigma 分布，让采样更偏向高噪声结构阶段，或者给低噪声细节阶段留下更多空间。
推荐测试顺序：

4.0 -> 5.0 -> 6.0 -> 8.0

经验判断：

目标	shift 倾向
静态画面细节、皮肤、纹理	低一点，约 3~5
大动作、强运动、镜头变化	高一点，约 5~8
当前问题 “时长/步数变高后细节倾向衰减”	先回到 4~5 验证

4.4 seed

可信度：源码可核验
seed 用于初始化随机噪声生成器：

seed 相同 + 参数相同 + 模型相同 -> 理论上结果可复现
seed 不同 -> 初始噪声不同，构图和细节可能变化很大

调参建议：

- 排查问题时固定 seed。
- 找到参数区间后改为随机 seed 批量挑结果。
- 不要只凭一个 seed 判断某个参数一定好或一定坏。

4.5 scheduler

可信度：源码可核验 + 使用经验
本机 WanVideoWrapper 支持多种调度器，例如 unipc、dpm++、euler、lcm 等。它们主要差异在于如何选 sigma 序列，以及如何做每一步的数值积分。
稳妥建议：

快速测试：euler 或 unipc
正式结果：unipc 或 dpm++
极少步数：lcm，但通常需要匹配对应加速 LoRA/蒸馏模型

5. VAE 与潜空间：哪些是源码事实，哪些是背景解释

5.1 VAE 的工作角色

可信度：背景知识 + 工作流可核验
VAE 是像素视频和 latent 视频之间的转换器：

输入图片/视频像素 -> VAE Encode -> latent
latent 采样完成 -> VAE Decode -> 输出图片/视频像素

为什么要用 latent：

直接在像素空间采样太大；
先压缩到 latent 空间，可以显著降低计算量和显存压力。

5.2 WanVideoWrapper 里可核验的 stride 常量

可信度：源码可核验

nodes_sampler.py 顶部有：

```
VAE_STRIDE = (4, 8, 8)
PATCH_SIZE = (1, 2, 2)
```

这表示典型 Wan 视频 latent 的时间/空间压缩关系常按：

```
时间方向约 4 倍
高度方向约 8 倍
宽度方向约 8 倍
```

注意：具体节点、模型版本、5B/14B、I2V/T2V、参考帧等场景可能有额外处理，所以文档里不再把某个 latent 尺寸写成绝对通用公式。

5.3 尺寸示例的正确读法

可信度：工作流推导，非源码通用定理

以 512x768、81 帧为例，空间上常见压缩为：

```
宽度 512 / 8 = 64
高度 768 / 8 = 96
```

时间上常见处理与节点具体实现有关，简单理解为：

```
81 帧视频进入视频 VAE 后，会变成远少于 81 个时间 latent 的序列。
```

这能解释为什么视频模型能处理多帧，但不能把它当成所有 Wan 工作流都固定等于 21 的绝对结论。

6. CLIP / T5 文本编码器：名字容易误导的地方

6.1 为什么节点叫 CLIPLoader，但 Wan 用的是 T5

可信度：工作流可核验

在当前 Wan 工作流里，文本编码器文件是：

```
umt5_xxl_fp8_e4m3fn_scaled.safetensors
```

虽然 ComfyUI 节点可能叫 CLIPLoader，但对 Wan 来说，它加载的是 Wan 兼容的文本编码器，常见是 UMT5/T5 系列，而不是传统 Stable Diffusion 里的 CLIP-L/CLIP-G 那套。

功能逻辑：

```
自然语言提示词
-> tokenizer 切成 token
-> T5/UMT5 编码成 text embeddings
-> UNet/Transformer 在采样每一步通过注意力读取这些 embeddings
```

6.2 正负提示词不是简单“加词/减词”

可信度：源码可核验 + 背景知识

正向提示词会进入 positiveembeds，负向提示词会进入 negativeembeds。采样时模型分别计算正向分支和负向分支，再用 CFG 公式合成。

也就是说：

```
正向提示词：告诉模型往哪里去
负向提示词：提供一个“不想要方向”的参照分支
CFG：控制正向分支相对负向分支被外推多少
```

因此负向提示词不是越长越好。如果负向里堆了大量宽泛质量词，可能把模型限制得太紧，反而影响细节和自然度。

7. 双 UNet / 高低噪声模型：如何谨慎理解

7.1 哪些是工作流事实

可信度：工作流可核验

你当前相关 I2V/WanAnimate 工作流中能看到两个模型：

```
wan2.2_i2v_low_noise_14B_fp8_scaled.safetensors
wan2.2_i2v_high_noise_14B_fp8_scaled.safetensors
```

并且常见连接方式是两段采样：

```
第一段采样：low_noise 模型
第二段采样：high_noise 模型
第一段保留 leftover noise
第二段继续接着去噪
```

7.2 哪些属于合理解释，而不是源码直接证明

可信度：工作流可核验 + 经验解释

合理解释是：

```
两段模型分别擅长不同噪声区间；
先处理整体结构、运动趋势；
再处理纹理、光影、局部细节。
```

但文档不再把“Low Noise 一定只负责骨架、High Noise 一定只负责皮肤细节”写成绝对事实。更准确的说法是：

```
两段模型在不同噪声区间工作，功能有倾向，但不是硬隔离。
```

7.3 LoRA 挂载到哪个阶段更合理

可信度：工作流可核验 + 调参经验

如果 LoRA 主要改变纹理、风格、局部细节，把它放在后段/细节段通常更稳。

如果 LoRA 改变角色整体形体、服装轮廓、构图习惯，可能需要同时测试：

```
只挂 high_noise
只挂 low_noise
两边都挂但 strength 降低
```

不要默认“所有 LoRA 都只挂 high_noise”。要看 LoRA 学到的是结构还是细节。

8. 关于“同参数多轮、步数/时长提高后效果衰减”的更可靠解释

8.1 已核验事实

可信度：源码/工作流可核验

你当前 T2V 测试脚本里关键参数是：

```
steps = 20
cfg = 5.0
shift = 8.0
seed = 777
LoRA strength = 0.8
frames = 81
```

已核验源码机制：

1. `cfg=5` 会把 `cond - uncond` 放大 5 倍。
2. `shift=8` 会让 `sigma` 在高噪声区间停留更久。
3. `seed=777` 固定时，初始噪声固定。
4. `frames` 增加会增加视频 `latent` 序列长度和时空注意力负担。

8.2 谨慎推断的原因链

可信度：推断，不是源码定理
更可信的解释链是：

```
cfg=5.0 偏高
-> 正负分支差值被强外推
-> 画面更容易过约束、发硬或细节不自然

shift=8.0 偏高
-> 高 sigma 区间停留更久
-> 低噪声细节修正集中在末尾

frames/时长增加
-> 同一模型要维持更多帧的一致性
-> 单帧局部细节可能被运动一致性和整体结构稳定性挤压

固定 seed 多轮比较
-> 只能说明同一个噪声起点下的趋势
-> 不能代表所有 seed 都会同样衰减
```

所以最先改的不是 LoRA，而是：

```
cfg: 5.0 -> 2.2 / 2.6 / 3.0
shift: 8.0 -> 4.0 / 5.0 / 6.0
seed: 先固定排查，再随机批量验证
```

8.3 推荐测试矩阵

可信度：调参建议
先固定 seed=777，只改 cfg 和 shift：

组别	steps	cfg	shift	目的
A	20	2.2	4.0	看细节是否恢复
B	20	2.6	4.0	稍微增强提示词控制
C	20	3.0	5.0	平衡控制和运动稳定
D	20	3.5	5.0	检查高一点 CFG 是否开始发硬
E	20	2.6	6.0	检查运动稳定是否改善

如果 A/B 细节明显好于原参数，就说明主要矛盾很可能是 CFG/shift 的组合过硬。然后再放开随机 seed，生成 5~10 个样本验证稳定性。

9. 专有名词速查表

名词	功能逻辑	可核验方式
latent	压缩后的图像/视频表示，采样主要在这里进行	看 VAE、sampler 的输入输出
sigma	当前噪声水平，越大越接近噪声	<code>`basic_flowmatch.py`</code>
shift	重映射 sigma 分布，让采样偏向不同噪声区间	<code>`basic_flowmatch.py`</code>
model_output	模型预测的更新方向	<code>`basic_flowmatch.py`</code> 的 <code>`step()`</code>
CFG	正负分支差值的外推强度	<code>`nodes_sampler.py`</code>
positive embeds	正向提示词编码结果	<code>`nodes_sampler.py`</code>
negative embeds	负向提示词编码结果	<code>`nodes_sampler.py`</code>
LoRA strength	LoRA 权重增量的乘法系数	<code>`custom_linear.py`</code>
VAE	像素和 latent 之间的编码/解码器	工作流节点 + VAE loader
T5/UMT5	Wan 使用的文本编码器	工作流 JSON / T5 loader
UNet/Transformer	执行去噪预测的主模型	model loader + sampler
scheduler	生成 sigma/timestep 并推进采样	<code>`wanvideo/schedulers/`</code>
seed	随机噪声种子	<code>`nodes_sampler.py`</code> 中 <code>`torch.Generator`</code>
fp8	模型权重量化精度，省显存但有精度权衡	模型名/loader 参数

10. 这版文档相对旧版的修正

旧版问题	本版处理
PDF 里 LaTeX 原样暴露，难读	全部改为代码块公式和表格解释
DDPM 背景公式和本机 Wan 实现混在一起	明确区分“背景知识”和“源码可核验”
shift 表数值不准确	按 <code>`basic_flowmatch.py`</code> 公式重新计算
“安全偏置”说法过于绝对	改为基于 CFG 公式的谨慎推断
双 UNet 分工写得像硬规则	改为“噪声区间倾向”，避免绝对化
VAE latent 尺寸写得过于固定	改为 stride 常量可核验 + 尺寸示例非通用定理

附录：最重要的源码等价片段

A. shift 重映射

```
self.sigmas = self.shift * self.sigmas / (1 + (self.shift - 1..
```

来源：ComfyUI/customnodes/ComfyUI-WanVideoWrapper/wanvideo/schedulers/basicflowmatch.py

B. Flow Matching 单步更新

```
prev_sample = sample + model_output * (sigma_ - sigma)
```

来源：ComfyUI/customnodes/ComfyUI-WanVideoWrapper/wanvideo/schedulers/basicflowmatch.py

C. add_noise

```
sample = (1 - sigma) * original_samples + sigma * noise
```

来源：ComfyUI/customnodes/ComfyUI-WanVideoWrapper/wanvideo/schedulers/basicflowmatch.py

D. CFG 合成

```
noise_pred = noise_pred_uncond_text + cfg_scale * (noise_pred..
```

来源: ComfyUI/customnodes/ComfyUI-WanVideoWrapper/nodessampler.py

E. LoRA 双矩阵增量

```
patch_diff = lora_diff_0 @ lora_diff_1
alpha = lora_diff_2 / lora_diff_1.shape[0]
scale = lora_strength * alpha
weight = weight + patch_diff * scale
```

来源: ComfyUI/customnodes/ComfyUI-WanVideoWrapper/customlinear.py

F. LoRA 单差分增量

```
weight = weight + lora_diff * lora_strength
```

来源: ComfyUI/customnodes/ComfyUI-WanVideoWrapper/customlinear.py